

GROUP - 66 Simple Multithreader Documentation

Steps to run on terminal:

1. Path of the file
2. Make
3. `./vector <numThreads> <size of vector>`
`./matrix <numThreads> <matrix dimension>`
4. Execution time and success message will be displayed

Implementation:

- `part_size()` splits the total elements into `<numThreads>` parts
In case of uneven split, the first `<extra>` threads get 1 extra element
- `split_range()` splits the range among `<numThreads>` threads using `part_size()` to determine the sub-ranges
- The structs `thread1D`, `thread2D` contain the the range for that thread and its corresponding lambda function
- `create_and_join()` (handles both 1D and 2D cases) - creates `<numThreads-1>` helper threads and waits for each thread to finish
The main thread executes the last part
- `start1D()`, `start2D()` call corresponding lambda functions for each element
- `std::chrono::steady_clock::now()` for execution time
- `parallel_for`
 1. Split range
 2. Assign each part to thread
 3. Create `<numThreads-1>` helper threads
 4. Main thread executes last part
 5. All threads concurrently run the lambda
 6. Wait for all threads to finish
 7. Print execution time
- Proper error handling implemented for invalid `numThreads`, `pthread_create`, `pthread_join`

Github repository link: <https://github.com/antelope0112/group66-multithreader>